

05. JavaScript 核心邏輯

一、關於 JavaScript

1. JavaScript 是什麼？

JavaScript 是讓網頁具有互動功能的程式語言，可以把它想像成網頁的大腦，負責控制各種動態效果與使用者互動，例如：

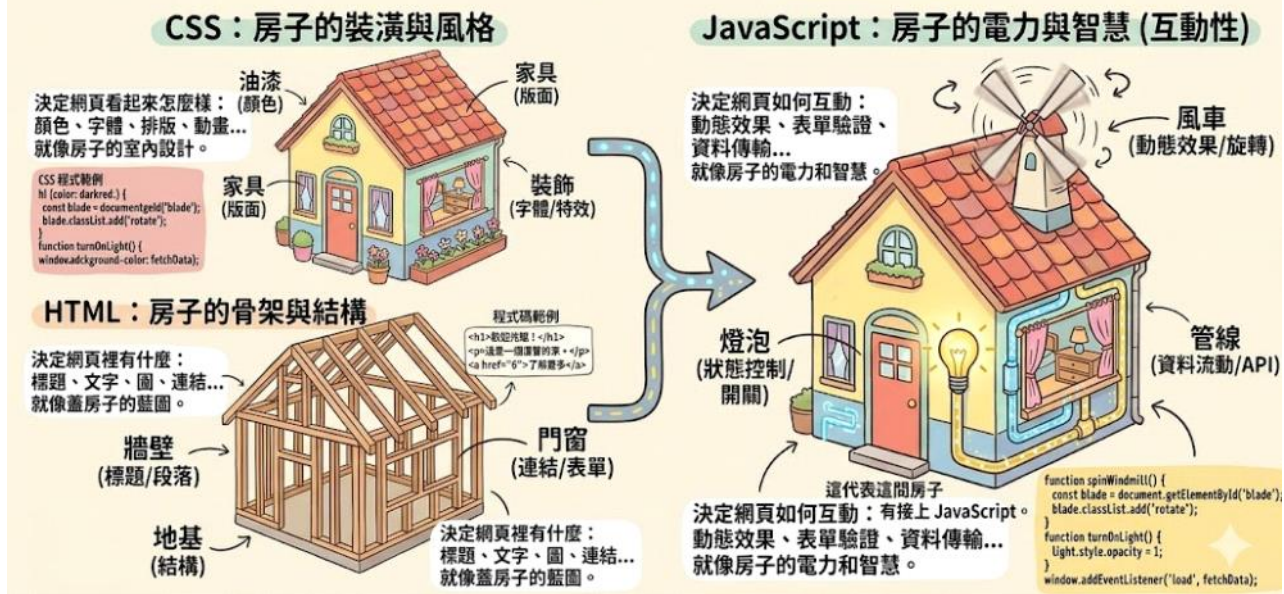
- 按下按鈕後顯示提示訊息
- 表單資料驗證
- 點擊選單展開或收合
- 圖片輪播
- 網頁動畫效果
- 即時更新資料
- 與伺服器交換資料 (API)
- 製作互動式網頁應用程式

※沒有 JavaScript，網頁仍然能正常顯示，但缺少互動與動態功能。

2. 如果把網站比喻成一棟房子：

- (1) HTML 是「鋼筋水泥與牆壁」，決定房子的結構與房間格局。
- (2) CSS 是「裝潢與家具配色」，決定房子美不美觀、牆壁是什麼顏色。
- (3) JavaScript 則是「智慧家電與水電系統」，當你按下開關，燈會亮（互動）；冷氣偵測到溫度過高，會自動啟動（邏輯）。
- (4) AI（人工智慧）則是你的「全能工頭助理」，你只要用「人類的語言」告訴他你想怎麼改裝水電，他就會幫你寫好施工藍圖（JS 程式碼），但你身為屋主，必須看得懂藍圖才能驗收。

網頁製作進階：HTML, CSS & JavaScript 輕鬆看！



3. Vibe Coding 前必備的 JavaScript 先備知識

在 AI 協作開發 (Vibe Coding) 的時代，不必親自手寫每一行程式碼，但必須具備「閱讀程式碼」與「精準指導 AI」的能力：

- 看懂結構：能一眼辨識 AI 生成的 ES6+ 現代語法，並找出潛在 Bug。
- 精準溝通：能用「陣列方法」與「非同步處理」的專業術語與 AI 對話，大幅減少溝通成本。
- 自主除錯：學會閱讀瀏覽器主機板 (Console) 的錯誤訊息，引導 AI 快速修復程式。

核心心智模型：你是導演，AI 是助理，在進入技術語法前，請先翻轉你的學習思維：

- 以前的學習法 (工程師思維)：死記
document.getElementById('myId').addEventListener(...) 每個字母拼法。
- Vibe Coding 學習法 (導演思維)：理解「網頁如何透過事件監聽器 (Event Listener) 來響應使用者的動作」，當 AI 給出這段程式碼時，你能一眼看出這是在處理「按鈕點擊事件」。

AI 預設會生成最現代的 JavaScript 語法。你不需要一字不漏手寫，但你要能「讀懂」它。

(1) 變數宣告：const 與 let 的區別

- const (常數)：宣告一個不能重新賦值的常數 (例如：固定設定值、API 網址)。
- let (變數)：宣告一個可以改變的變數 (例如：計數器、遊戲分數)。

※Vibe 導演提示：如果 AI 把一個需要改變的變數宣告為 const，瀏覽器會報錯 Assignment to constant variable。此時你就能一秒抓出 Bug，告訴 AI：「變數宣告錯誤，請改用 let」。



JavaScript 變數



const、let 與 var 的使用重點

1 const (常數)



宣告一個不能重新賦值的常數

適合固定設定值、API 網址、不可變更的參考資料

```
> const API_URL =  
  'https://api.example.com';
```



不能重新指定

2 let (變數)



宣告一個可以改變的變數

適合計數器、遊戲分數、會變動的資料

0 1 0

```
> let score = 0;  
  score = score + 10;
```



可以重新賦值

3 不要使用 var



現代 JavaScript 開發中，建議以 **const** 與 **let** 取代 **var**

- 容易造成作用域混亂
- 變數提升 (hoisting) 較難理解
- 可讀性與維護性較差



快速記憶

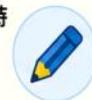
1 預設優先使用

const



2 需要改值時再用

let



3 避免使用

var



(2) 箭頭函式 (Arrow Functions)

現代 JS 幾乎都用簡短的箭頭函式來包裝「功能模組」。

// 傳統寫法

```
function sayHello(name) {  
  return "Hello " + name;  
}
```

// 現代寫法 (AI 最愛用)

```
const sayHello = (name) => `Hello ${name}`;
```

※如何解讀：看到 => 就要知道這是一個「功能區塊 (函式)」，它接受左邊括號裡的輸入 (參數)，並回傳右邊的結果。

(3) 解構賦值 (Destructuring) 與 樣板字串 (Template Literals)

- 解構賦值：快速從複雜的物件或陣列中拿出資料。

```
const user = { name: "Alex", age: 25 };
```

```
const { name, age } = user; // 快速提取變數，不需寫 user.name 和 user.age
```

- 樣板字串：用反單引號 `` 來動態組合字串，裡面用 \${變數}。

```
const message = `哈囉 ${name}，你今年 ${age} 歲。`;
```

(4) 資料的變形術 — 陣列方法 (Array Methods)

網頁上 90% 的資料都是以「列表 (陣列)」的形式存在，例如商品列表、待辦事項、留言板。AI 非常喜歡用以下三種方法來處理與渲染這些資料：



JavaScript 陣列常用方法

map()、filter()、reduce() 一次看懂

1



map() — 投影與渲染

 概念：把陣列中的每一筆舊資料，轉換成新資料。

 應用：在 React 中常用來將「純文字資料」轉換成「HTML 畫面元素」。

[1 , 2 , 3] → [2 , 4 , 6]

每一筆都被轉換

2



filter() — 篩選

 概念：根據條件，只留下符合的資料。

 例如：篩選出「未完成」的待辦事項。

[已完成 , 未完成 , 未完成] → [未完成 , 未完成]

只留下符合條件的資料

3



reduce() — 累加

 概念：將資料加總或整合成單一結果。

 例如：計算購物車的商品總金額。

[100 , 250 , 80] → 430

把多筆資料累積成一個結果



快速記憶

- map()：轉換資料
- filter()：篩選資料
- reduce()：累積結果

※ Vibe 導演提示：當你想讓 AI 做出篩選功能時，直接使用專業術語。

- ◆ 高階指令：「請幫我用 filter 篩選出大於 100 元的商品，再用 map 渲染到畫面上。」這樣下指令，AI 一次就能給出極其精準、乾淨的程式碼。

(5) 非同步程式設計 — 與外部世界連線 (Async / Await)

現代網頁幾乎都需要「接 API」抓取外部資料（例如天氣、地圖、後端資料庫）。這是 Vibe Coding 中最容易遇到 Bug、最需要人工介入檢查的環節。

1) 什麼是非同步 (Asynchronous)？

JavaScript 是單執行緒 (Single Thread) 的。當我們向伺服器請求資料時，不能讓整個網頁卡住等待伺服器回應。因此我們需要「非同步」處理，讓網頁一邊載入，一邊在背景等資料。

2) 核心結構 (精準閱讀範本)：

```
const fetchData = async () => {
  try {
    const response = await fetch('https://api.example.com/user');
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error("抓取失敗：", error);
  }
};
```

JavaScript 非同步程式設計

async、await、try...catch 一次看懂

1 async — 宣告非同步函式

- 概念：在函式前加上 `async`，表示這個功能裡面可能有需要等待的步驟。
- 作用：讓函式可以搭配 `await` 使用，適合處理 API 請求、資料庫讀取等流程。

```
const loadData = async () => {  
  ...  
}
```

先告訴 JavaScript：這裡有非同步流程

2 await — 等待結果再往下執行

- 概念：遇到 `await` 時，JS 會先暫停這段函式，等資料完成後再繼續。
- 常見用途：等待 `fetch()`、資料庫查詢、檔案讀取等非同步結果。

發送請求 → 等待回應 → 取得資料

```
const data = await fetchData();
```

等這行完成，後面程式才繼續

3 try...catch — 錯誤處理與防呆

- 概念：把可能出錯的程式放進 `try`，若失敗就交給 `catch` 處理。
- 重要性：當 API 壞掉或網路斷線時，程式不會直接當機，而是能優雅顯示錯誤訊息。

成功取得資料

```
try {  
  const data = await fetchData();  
} catch (error) {  
  console.log(error);  
}
```

發生錯誤 / 顯示錯誤訊息

防禦性程式設計的關鍵



快速記憶

1 async：

宣告「這裡有非同步流程」

2 await：

等待完成再往下

3 try...catch：

發生錯誤也能安全處理

※Vibe 導演提示：如果 AI 生成的 API 串接程式沒有寫 `try...catch`，網頁一旦出錯就會直接當機。你可以主動提醒 AI：「請為這段 `fetch` API 加上 `try...catch` 與 `loading` 狀態。」

(6) 瀏覽器與使用者的橋樑 — DOM 操作與事件 (Events)

網頁能動，是因為瀏覽器能監聽並響應使用者的行為。



JavaScript DOM 操作與事件

DOM、選取元素、修改內容、事件監聽 一次看懂

1 DOM (Document Object Model)

瀏覽器把 HTML 轉成可控制的樹狀物件

- 概念：瀏覽器把 HTML 結構轉化為 JS 可以控制的樹狀物件。
- 重點：JS 可以找到節點、修改內容、改變樣式。

HTML 文件

```
<html>
<body>
  <h1>標題</h1>
  <p>段落</p>
  <button class="btn">
    按鈕
  </button>
</body>
</html>
```

DOM 樹狀結構

```
graph TD
  document --> html
  document --> body
  html --> head
  html --> body
  body --> h1
  body --> p
  body --> button_btn[button.btn]
```

JS 可控制

JS 可以存取、修改、操作節點

2 選取元素與修改內容

A 選取元素

- document.querySelector('.btn')：像 CSS 選擇器一樣方便

```
const btn = document.querySelector('.btn');
```

B 修改內容

- element.textContent = '新文字'：改文字內容
- element.classList.add('active')：新增 class 樣式

```
btn.textContent = '新文字';
btn.classList.add('active');
```

修改前

按鈕

↓

修改後

按鈕

(新增 active class)

3 事件監聽 (Event Listeners)

當使用者做某事時，觸發特定的 JS 程式碼

- 常見事件：click、submit、input、keydown
- 做法：用 addEventListener() 監聽事件

```
btn.addEventListener('click', () => {
  btn.textContent = '已點擊';
});
```

click 滑鼠點擊

submit 表單提交

input 輸入內容

keydown 鍵盤按下

使用者動作 → 事件觸發 → 執行 JS

★ 快速記憶

1 DOM：把網頁變成 JS 可操作物件

2 querySelector()：選取元素

3 addEventListener()：監聽使用者操作

當使用者做某事時，觸發特定的 JS 程式碼。

- 常見事件：click (點擊), submit (表單送出), input (輸入中), keydown (按下鍵盤)。

※ **Vibe 導演提示**：設計不同裝置的網頁時，要引導 AI 監聽對應的事件。例如想做手機版網頁，你可以提示 AI：「請把 click 事件改為監聽 touchstart 與 touchend 以支援流暢的手勢。」

(7) 狀態管理 (State) 的思維

無論你用純 JS 還是現代前端框架 (如 React、Vue)，「狀態 (State)」都是核心。

- 狀態是什麼？：驅動畫面改變的關鍵資料。例如：購物車裡的商品數量、切換深色/淺色模式的開關。
- 資料驅動 UI 的觀念：不要手動去修改每一個 HTML 標籤。正確的心態是：修改狀態 (資料)，讓畫面根據狀態自動重新渲染。

※**Vibe 導演提示**：對 AI 下指令時，改用「狀態」的思維去溝通。

- 低效指令：「當我點這個按鈕時，把這個文字改成紅色，那個框框隱藏起來。」
- 高效指令：「請幫我建立一個名為 isDarkMode 的狀態，當它為 true 時，自動給 body 加上 dark 類別，並連動切換按鈕的文字。」

(8) Vibe Coding 的除錯救星：看懂主機板 (Console) 錯誤訊息

當網頁沒反應時，第一步永遠是按 **F12** (Mac 為 Cmd + Option + I) 打開「開發者工具 (DevTools)」，查看 **Console (主機板/主控台)**。

以下是常見的三大紅色錯誤訊息，看懂它們你就能精確「對症下藥」指導 AI：



Vibe Coding 的除錯救星



JS Vibe Coding 的除錯救星 看懂 Console 錯誤訊息

1 ReferenceError: X is not defined



意思
程式碼嘗試使用一個不存在，或還沒有宣告的變數 / 函式 X。

對 AI 的指令
我的 Console 錯誤訊息顯示 X 未定義，請檢查是不是漏了宣告或匯入 X 模組？

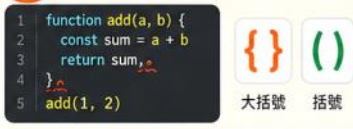
2 TypeError: Cannot read properties of undefined (reading 'map')



意思
你嘗試對一個空值 (undefined / null) 做事情。通常是因為 API 還沒回傳資料，程式就急著去讀取並渲染。
防禦性判斷 更安全！

對 AI 的指令
我在載入時遇到 Cannot read properties of undefined，請在渲染列表前加上防禦性判斷 (例如 Optional Chaining 或 loading 檢查)。

3 SyntaxError: Unexpected token



意思
語法錯誤，例如少了大括號、括號沒對齊，或少打逗號。通常發生在手動複製貼上、拼接程式碼時。

對 AI 的指令
我的程式碼出現 SyntaxError，請幫我檢查並修復程式碼中可能缺失的大括號或標點符號。

快速記憶

- 1. ReferenceError: 先檢查有沒有宣告 / 匯入
- 2. TypeError: 先檢查資料是不是 undefined
- 3. SyntaxError: 先檢查括號、逗號、符號

(9) 總結：prompt實戰溝通指南

模糊的低效指令 (新手)	高效的「Vibe 指令」(懂 JS 概念的導演)
「幫我做一個可以按的清單。」	「幫我用一個陣列儲存資料，並用 map 渲染成列表。點擊刪除按鈕時，觸發事件並用 filter 過濾掉該項目，最後更新 State。」
「網頁載入時要出現資料，但現在畫面一片空白。」	「我的 fetch API 可能是非同步問題，請幫我加上 async/await，並在資料還沒載入成功前，先顯示一個 Loading 動畫。」
「按鈕按了沒反應，怎麼辦？」	「控制台報錯顯示 TypeError: ... is not a function，請幫我檢查按鈕的 onClick 事件監聽器綁定是否正確。」

二、變數 (Variable)

1. 核心觀念

- 變數宣告：告訴電腦我們要保留一塊記憶體空間來存放資料。
- 作用域 (Scope)：
 - let 與 const 具有區塊作用域 (Block Scope)，只在 {} 內有效。
 - var 具有函式作用域 (Function Scope)，容易引發「變數污染」或「變數提升 (Hoisting)」等問題。
- 命名規範：小駝峰命名法 (camelCase)，例如 userAge、totalCartPrice。

2. 生活化案例

變數就像是「貼有標籤的收納盒」：

- let (可變變數)：像是「存錢筒」，裡面的金額可以隨時放入或取出改變。

- (2) `const` (常數)：像是「身分證字號」，一旦寫上去封存，就再也不能修改 (唯讀)。
- (3) `var` (舊式變數)：像是「沒有蓋子的舊紙箱」，容易翻倒而且容易跟別的箱子混淆 (不安全)。

3. Prompt 範例

初階	「請解釋 JavaScript 當中 <code>let</code> 和 <code>const</code> 的差別，並各舉一個適合的使用情境。」
進階	「我寫了一段舊的 JS 程式碼，裡面充滿了 <code>var</code> 。請幫我將它們重構為現代的 <code>let</code> 和 <code>const</code> ，並解釋你為何做這樣的更換，以及如何避免變數提升 (Hoisting) 的安全漏洞。」

4. 語法解析

```
let memberPoints = 100; // 1. 宣告一個名為 memberPoints 的變數，並放入初始值 100
memberPoints = 150;      // 2. 重新設定變數的值為 150 (不需再寫 let)

const taxRate = 0.05;    // 3. 宣告一個常數 taxRate，放入值 0.05
// taxRate = 0.1;        // ❌ 錯誤！常數不可重新賦值，這行會導致報錯 (Assignment to constant variable)
```

```
// 變數宣告與計算實作
const shopName = "星空雜貨店"; // 店名不變，用 const
let visitorCount = 0;           // 訪客數會變，用 let
let currentDiscount = 0.9;      // 折扣率可能變動，用 let

visitorCount = visitorCount + 1; // 迎來第一位客人
console.log("店名：" + shopName);
console.log("今日訪客人數：" + visitorCount);
console.log("目前折扣：" + (currentDiscount * 10) + " 折");
```

5. AI 常見錯誤

- (1) **AI 輸出過時語法**：有時 AI 會無意識地使用 `var` 來宣告變數，這在現代前端開發中是不被推薦的。
- (2) **解決方案**：在提示詞中加上限制，例如：「請一律使用 ES6+ 規範，使用 `let` 與 `const` 進行變數宣告，嚴禁使用 `var`。」

三、資料型別 (Data Type)

1. 核心觀念

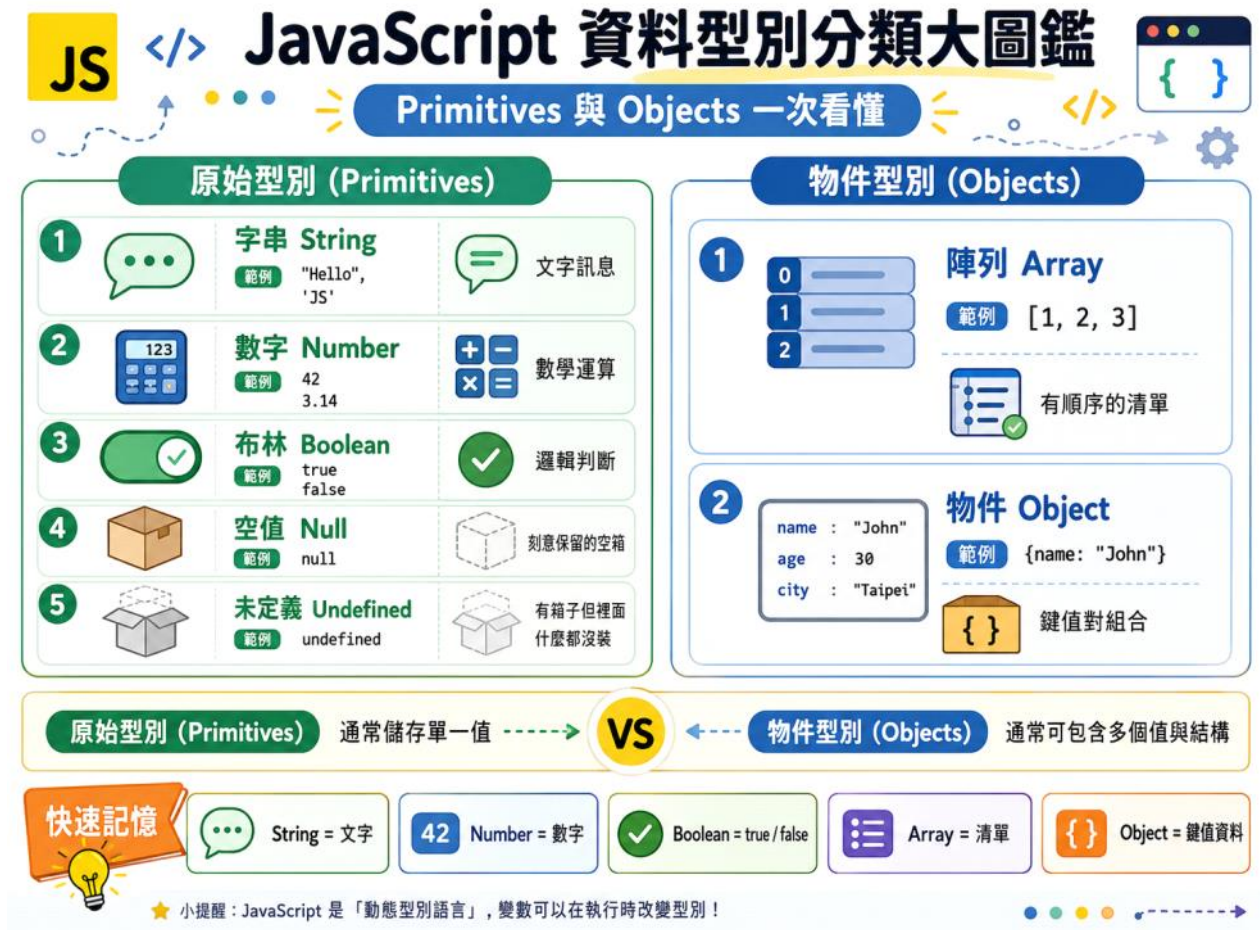
- (1) **基本型別 (Primitive Types)**：String, Number, Boolean, null, undefined, Symbol, BigInt。值直接存在記憶體 (Stack) 中。
- (2) **物件型別 (Object Types)**：Object, Array, Function。存放在 Heap 中，變數保存的是指向該物件的「參照位址 (Reference)」。

- (3) **動態型別 (Dynamic Typing)**：JS 變數不需要事先宣告型別，變數的型別是由「放進去的值」決定的。

2. 生活化案例

資料型別就像是廚房裡不同的容器：

- (1) 湯匙用來喝湯，不能用來裝大西瓜（型別不合）。
- (2) **字串 (String)**：文字，像是一串珍珠項鍊（用雙引號或單引號包起來）。
- (3) **數字 (Number)**：計算用的數值（包含整數與浮點數）。
- (4) **布林值 (Boolean)**：開關，只有「開 (true)」和「關 (false)」。
- (5) **物件 (Object) 與陣列 (Array)**：置物櫃，可以把很多不同種類的東西分類收納在一起。



3. Prompt 範例

初階	「在 JavaScript 中，如果我把一個數字和一個字串相加，會發生什麼事？請舉例說明什麼是隱式轉型 (Implicit Coercion)。」
進階	「請解釋 JavaScript 中 null 與 undefined 的本質差異，並提供一段程式碼，展示在非嚴格比較 () 與嚴格比較 (=) 下，這兩者與其他型別的比較結果。另外，請說明為什麼 typeof null 的結果會是 'object'？」

4. 語法解析

```
let productName = "蘋果"; // String 型別，用雙引號包裹
let price = 30;           // Number 型別，直接寫數字
let isOnSale = true;      // Boolean 型別，只有 true 或 false

console.log(typeof productName); // 輸出 "string"
console.log(typeof price);       // 輸出 "number"

// 隱式轉型的可怕陷阱：
let result = "30" + 5; // "30" 是字串，JS 會把 5 也轉成字串，結果變成 "305"!
console.log(result);   // 輸出 "305"
```

```
// 資料型別與顯式轉型實作
let inputPrice = "199"; // 模擬從網頁輸入框拿到的「字串」
let shippingFee = "60"; // 模擬字串運費

// ❌ 錯誤寫法：console.log(inputPrice + shippingFee); -> 輸出 "19960"

// 正確寫法：使用 Number() 轉換型別
let total = Number(inputPrice) + Number(shippingFee);

console.log("商品價格型別：" + typeof inputPrice);
console.log("轉換後的總金額：" + total); // 輸出 259
console.log("總金額型別：" + typeof total); // 輸出 "number"
```

5. AI 常見錯誤

- 型別混亂導致 NaN 或運算錯誤：AI 有時會直接拿字串型別的 API 回傳值（例如 input 的 value）進行數學加法，導致網頁呈現 10020 而不是 120。
- 解決方案：要求 AI 進行數值運算前，先做顯式型別轉換。例如使用 Number() 或 parseInt()。

四、Function 函式

1. 核心觀念

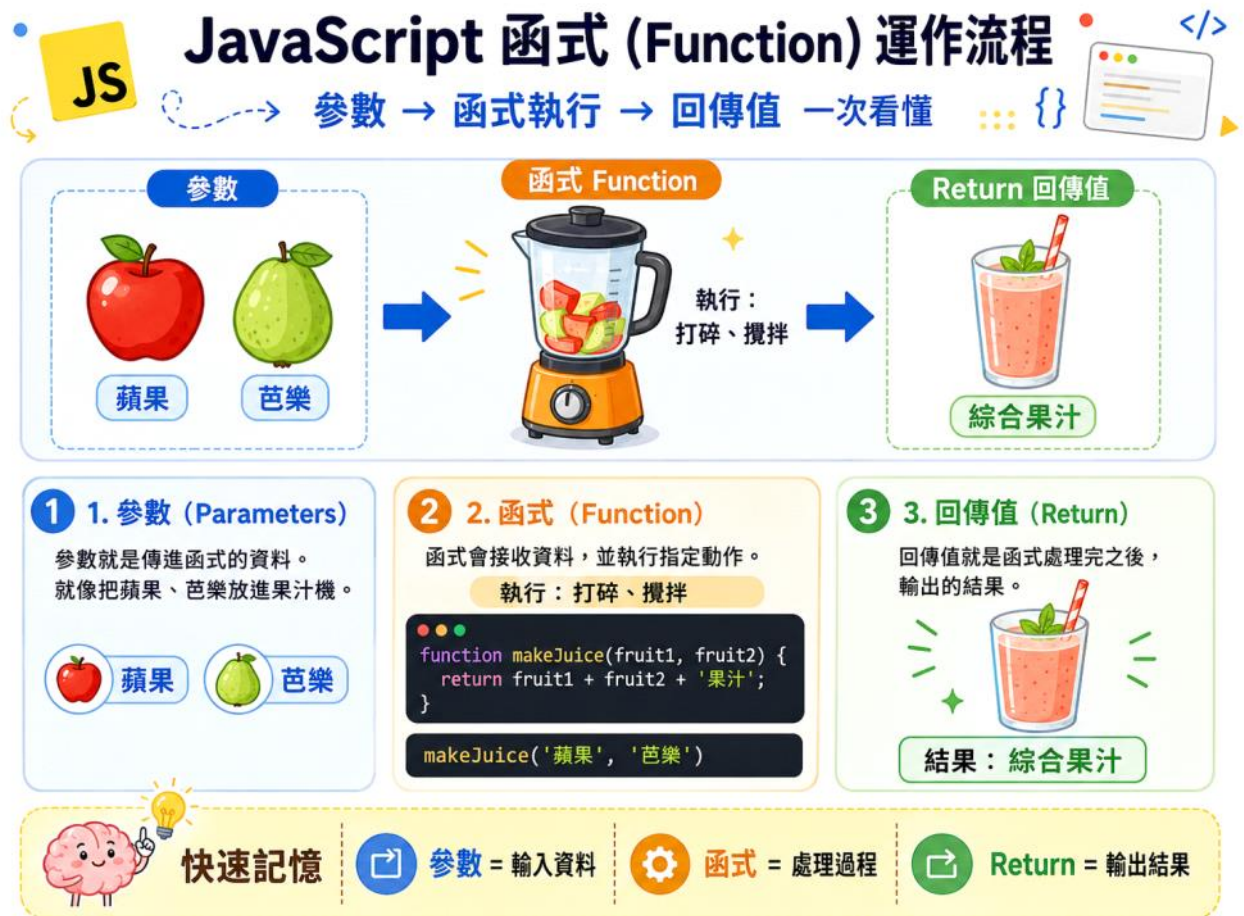
- (1) 重複使用性 (Reusability)：將常用程式碼打包，避免重複撰寫（DRY - Don't Repeat Yourself）。
- (2) 回傳值 (Return Value)：return 關鍵字會結束函式的執行，並將結果送回呼叫它的地方。若無寫 return，預設回傳 undefined。
- (3) 箭頭函式 (Arrow Functions)：ES6 語法糖，寫法更簡潔，且不綁定自己的 this。

2. 生活化案例

函式就像是一台「果汁機」：

- (1) 輸入 (Input)：你放進去的「水果」（參數）。
- (2) 處理 (Process)：果汁機內部的刀片旋轉（函式主體程式碼）。
- (3) 輸出 (Output)：倒出來的「果汁」（回傳值）。

你可以重複使用這台果汁機，每次放不同的水果，就能得到不同的果汁。



3. Prompt 範例

初階	「請幫我寫一個 JavaScript 函式，輸入一個攝氏溫度，會自動計算並回傳華氏溫度 (公式為： $F = C \times 1.8 + 32$)。」
進階	「請幫我將以下傳統函式宣告重構為 ES6 箭頭函式，並向我解釋箭頭函式在隱式回傳 (Implicit Return) 以及 this 綁定上的特性與限制。」

4. 語法解析

```
// 1. 傳統函式宣告
function makeJuice(fruit1, fruit2) {
  // fruit1, fruit2 是參數 (收納果實的箱子)
  let juice = fruit1 + "混搭" + fruit2 + "汁";
  return juice; // 將結果丟回給呼叫者，並結束函式
}

// 2. 呼叫函式
let myDrink = makeJuice("蘋果", "鳳梨"); // 傳入引數 "蘋果"、"鳳梨"
console.log(myDrink); // 輸出 "蘋果混搭鳳梨汁"

// 3. 箭頭函式寫法 (簡潔版)
const add = (a, b) => a + b; // 若只有一行，可省略大括號與 return
console.log(add(5, 10)); // 輸出 15
```

```
// 計算折扣價的函式
function calculateDiscountPrice(originalPrice, discountRate) {
  // 安全防護：如果輸入的不是數字，就回傳 0
  if (typeof originalPrice !== "number" || typeof discountRate !== "number") {
    return 0;
  }
  let finalPrice = originalPrice * discountRate;
  return Math.round(finalPrice); // 四捨五入並回傳
}

// 呼叫函式測試
let price1 = calculateDiscountPrice(1000, 0.85);
let price2 = calculateDiscountPrice(450, 0.7);

console.log("商品 A 折扣價：" + price1); // 850
console.log("商品 B 折扣價：" + price2); // 315
```

5. AI 常見錯誤

- 忘記寫 **return**：AI 有時寫完複雜的處理邏輯後，卻漏掉了 **return**，導致外部接收到的是 **undefined**。
- 解決方案：在提示詞中強調：「請確保函式處理完畢後，明確回傳計算結果，並附上 **return** 語句。」

五、條件判斷 (if / switch)

1. 核心觀念

- (1) **If...Else 語法**：適合範圍型判斷 (例如：分數 > 90、金額 < 1000)。
- (2) **Switch 語法**：適合單一變數、多個「等值」分支的判斷。
- (3) **嚴格比對**：switch 的比對是採用嚴格相等 (===)，型別不同就不會匹配。
- (4) **邏輯運算子**：
 - 1) && (And)：所有條件皆為 true 才是 true。
 - 2) || (Or)：只要有一個條件為 true 就是 true。
 - 3) ! (Not)：把 true 變 false，false 變 true。

2. 生活化案例

條件判斷就像是「十字路口的紅綠燈」與「路線指引」：

- (1) **If / Else**：如果今天是雨天，我就帶傘；否則，如果今天是陰天，我帶遮陽帽；其餘情況，我什麼都不帶。
- (2) **Switch**：你站在自動售票機前，按「按鈕 A」就出 A 車票，按「按鈕 B」就出 B 車票，如果按到沒設定的按鈕，就退幣 (Default)。

JS JavaScript 條件判斷 (if / switch)

根據條件，決定程式要走哪一條路



1 if 判斷

if 會先檢查條件是否成立。條件成立，就執行對應程式碼。

```

if (amount >= 2000) {
  shipping = 0;
} else {
  shipping = 60;
}
  
```



適合：條件分支少、邏輯直接。

2 switch 判斷

switch 適合拿來處理多個固定選項。

```

switch (level) {
  case 'VIP':
    discount = 20;
    break;
  case 'Member':
    discount = 10;
    break;
  default:
    discount = 0;
}
  
```



適合：多種固定選項，例如會員等級、狀態分類。



快速記憶

1 if：判斷單一條件

2 else：條件不成立時執行

3 switch：多選項分類判斷

3. Prompt 範例

初階	「請幫我寫一個 JavaScript 的 if-else 邏輯，輸入學生的分數，90 分以上印出『優等』，80~89 分印出『甲等』，70~79 分印出『乙等』，低於 70 分印出『需再加油』。」
進階	「我需要設計一個電商結帳折扣計算邏輯。規則包含：1. 滿 3000 元折 300 元 2. 如果是 VIP 會員再享 95 折 3. 如果使用特定信用卡結帳再折 50 元。請幫我用最乾淨、沒有巢狀 (Nested) 過深問題的 JavaScript 程式碼實現，並使用衛述句 (Guard Clauses) 來優化邏輯結構。」

4. 語法解析

```

let totalAmount = 1500;
let isMember = true;

// 1. If-Else 多重條件判斷
if (totalAmount >= 2000 && isMember) {
  console.log("恭喜！您享有 8 折優惠且免運費。");
} else if (totalAmount >= 1000 || isMember) {
  console.log("您享有 9 折優惠！");
} else {
  console.log("很抱歉，您無折扣優惠。");
}

// 2. Switch 條件判斷
let payMethod = "LINE_PAY";
switch (payMethod) {
  case "CASH":
    console.log("現金付款，無額外點數回饋。");
    break; // 務必加上 break，否則會發生 fall-through (繼續往下執行)
  case "LINE_PAY":
  case "APPLE_PAY":
    console.log("行動支付，享有 2% 點數回饋！");
    break;
  default:
    console.log("不支援此付款方式。");
}

```

```

// 綜合折扣判定系統
function checkout(cartTotal, isVip, couponUsed) {
  let finalTotal = cartTotal;

  // 邏輯 1: 如果是 VIP 且消費滿 1000
  if (isVip && cartTotal >= 1000) {
    finalTotal = cartTotal * 0.8; // 打 8 折
    console.log("觸發 VIP 滿千優惠！");
  } else if (isVip || couponUsed === "SAVE100") {
    finalTotal = cartTotal - 100; // 折 100 元
    console.log("觸發一般會員或優惠券折百優惠！");
  } else {
    console.log("無觸發任何折扣。");
  }

  return finalTotal < 0 ? 0 : finalTotal; // 防止折扣完變負數
}

console.log("應付金額：" + checkout(1200, true, "NONE") + " 元"); // 960
console.log("應付金額：" + checkout(500, false, "SAVE100") + " 元"); // 400

```

5. AI 常見錯誤

- Switch 忘記寫 break：這是經典的 Bug。AI 有時在寫 switch 時，漏寫了 break，導致執行完第一個匹配的區塊後，又一路往下執行到 default。
- 解決方案：審查 AI 程式碼時，特別檢查每個 case 後面是否有搭配 break；或者在提示詞中

寫明：「請使用 switch 語法，並注意每個 case 的 break 完整性。」

六、DOM 事件監聽 (Click/Submit)

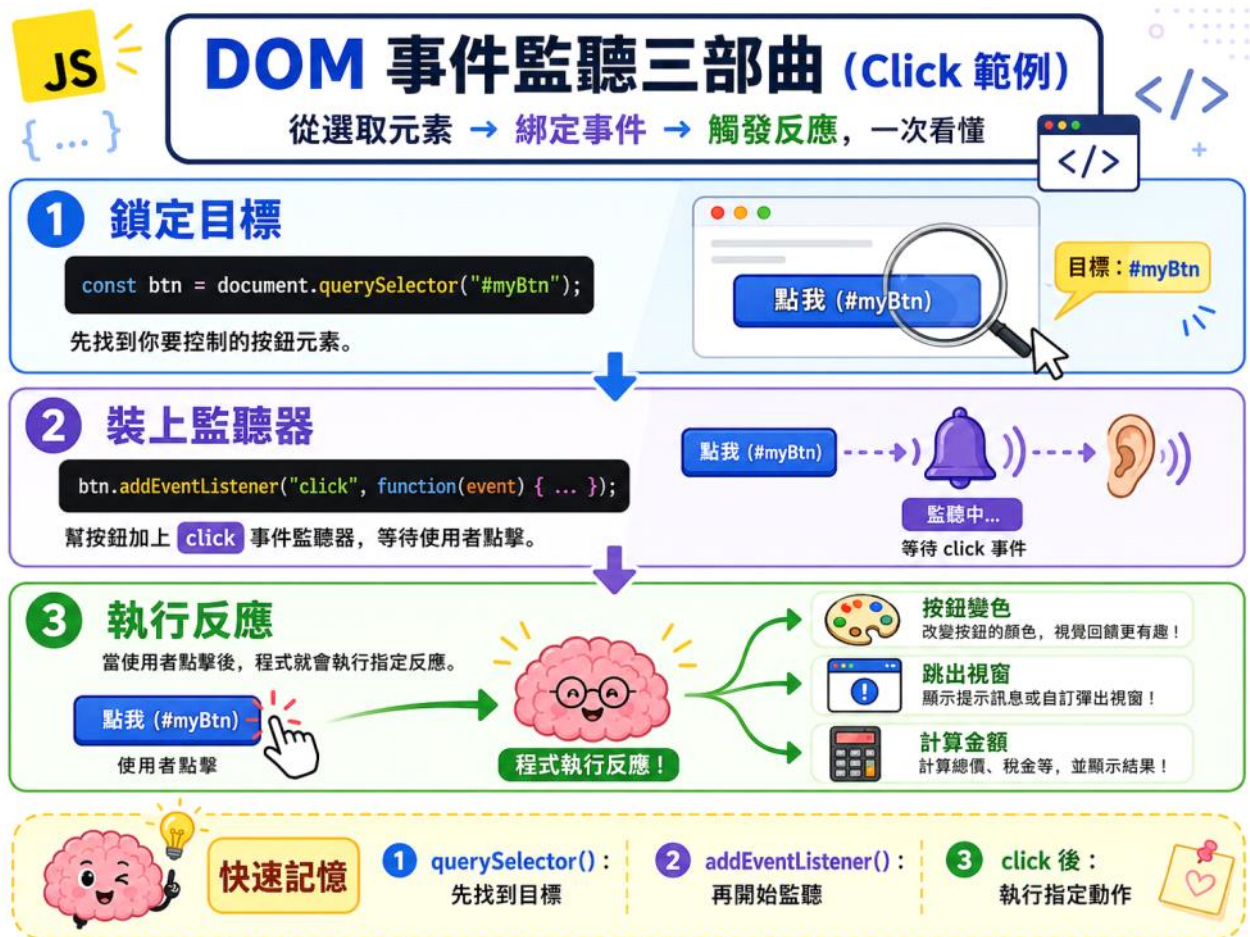
1. 核心觀念

- (1) **DOM (文件物件模型)**：瀏覽器將 HTML 轉譯成樹狀結構，讓 JavaScript 可以動態修改它。
- (2) **事件物件 (Event Object, 縮寫為 e 或 event)**：當事件發生時，瀏覽器會自動產生一個物件，裡面包含事件的所有細節（如：誰被點擊了、按了什麼鍵）。
- (3) **表單送出 (Submit)**：表單預設在點擊 submit 按鈕後會「重新整理網頁」。在單頁應用（SPA）或 Ajax 開發中，我們必須使用 `e.preventDefault()` 來阻止這項預設行為，改用 JS 處理資料。

2. 生活化案例

事件監聽就像是「門鈴與警報系統」：

- (1) 網頁按鈕就是「門鈴按鈕」。
- (2) JavaScript 就是「門鈴感應器」。
- (3) `addEventListener("click", ...)` 就是設定：「當有人按（click）門鈴時，就播放音樂（執行函式）」。
- (4) `e.preventDefault()` 就像是「把自動郵件的寄出按鈕攔截下來」，先檢查裡面裝的是不是詐騙信件，確認沒問題才放行。



3. Prompt 範例

初階	「請給我一段簡單的 HTML 與 JavaScript，畫面上有一個按鈕。當使用者點擊按鈕時，網頁背景顏色會隨機變成紅色、綠色或藍色。」
進階	「請寫出一段健全的 JS 程式碼，監聽一個 id 為 login-form 的表單 submit 事件。要求： 1. 使用 preventDefault() 阻止頁面跳轉。 2. 獲取表單內 username 和 password 輸入框的值。 3. 進行基礎驗證（密碼必須大於 6 位數），若不通過則在畫面上動態呈現紅色警告文字，通過則在 console 輸出成功登入資訊。」

4. 語法解析

```
// 1. 取得 HTML 中的按鈕元素
const submitButton = document.querySelector("#submit-btn");

// 2. 幫按鈕加上點擊事件監聽器
submitButton.addEventListener("click", function(event) {
  // event 參數代表這次「點擊」的事件物件
  console.log("按鈕被點擊了!");
  console.log("被點擊的元素是:", event.target); // 輸出被點擊的 HTML 節點
});
```

```

<!DOCTYPE html>
<html lang="zh-TW">
<head>
  <meta charset="UTF-8">
  <title>事件監聽實作</title>
  <style>
    .error { color: red; font-weight: bold; }
  </style>
</head>
<body>
  <form id="membership-form">
    <label for="username">會員名稱 :</label>
    <input type="text" id="username" placeholder="請輸入姓名">
    <button type="submit">加入會員</button>
  </form>
  <p id="msg-box"></p>

  <script>
    const form = document.querySelector("#membership-form");
    const msgBox = document.querySelector("#msg-box");

    form.addEventListener("submit", function(event) {
      event.preventDefault(); // 核心！阻止表單跳轉與網頁重整

      const nameInput = document.querySelector("#username").value.trim();

      if (nameInput === "") {
        msgBox.textContent = "❌ 請輸入姓名，欄位不可為空！";
        msgBox.className = "error";
      } else {
        msgBox.textContent = "👋 歡迎你," + nameInput + "! 成功註冊會員！";
        msgBox.className = ""; // 清除 error 樣式
      }
    });
  </script>
</body>
</html>

```

5. AI 常見錯誤

- 忘記阻止表單預設送出行為：AI 寫表單送出邏輯時，常常忘了加 `e.preventDefault()`，導致畫面一閃就自動重新整理了，所有資料全部消失。
- 解決方案：在處理 submit 事件的提示詞中，明確規定：「請確保在 submit 事件處理器中，第一行就加入 `e.preventDefault()` 以阻止網頁重新整理。」

七、陣列處理

1. 核心觀念

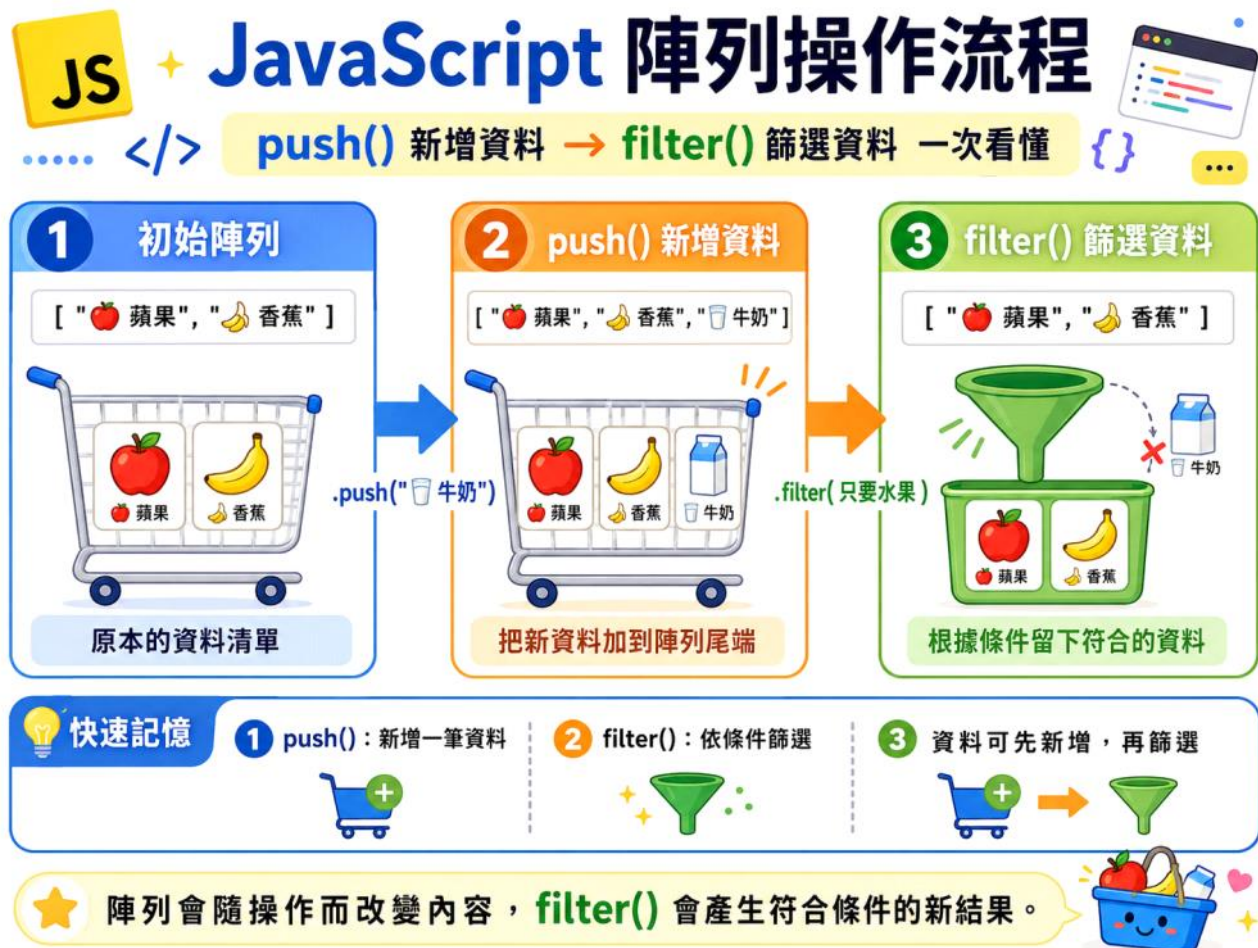
- (1) 索引值 (Index)：陣列中每個元素都有編號，從 0 開始。
- (2) 常用方法：
 - 1) `push()`：在陣列末端新增元素。
 - 2) `pop()`：刪除並回傳陣列末端的元素。

- 3) `forEach()`：逐一巡迴走訪陣列，不會產生新陣列。
- 4) `map()`：加工陣列中每個元素，並回傳一個全新的陣列。
- 5) `filter()`：篩選符合條件的元素，並回傳一個全新的陣列。

2. 生活化案例

陣列就像是「超市購物車」：

- (1) **陣列宣告**：你推了一輛空的購物車 `let cart = []`。
- (2) **push**：你把一瓶「牛奶」放進車尾。
- (3) **pop**：你想想不對，把車尾最後放進去的那盒「巧克力」拿出來退掉。
- (4) **forEach**：收銀員一件件掃描購物車裡的所有商品，計算總金額。
- (5) **filter**：你想把購物車裡「價格超過 500 元的奢侈品」挑出來。



3. Prompt 範例

初階	「請寫出一段 JavaScript 程式碼，建立一個名為 <code>scores</code> 的數字陣列。接著使用 <code>filter</code> 方法篩選出所有大於等於 60 分的成績。」
進階	「我有一個包含商品物件的陣列： <code>let products = [{name: 'A', price: 100}, {name: 'B', price: 250}, {name: 'C', price: 80}]</code> 。請幫我寫出一段程式碼：1. 先用 <code>filter</code> 挑出價格大於 90 元的商品。2. 再用 <code>map</code> 將挑選出的商品價格打 9 折（乘上 0.9）。3. 最後使用 <code>forEach</code> 把結果印在 <code>console</code> 中。請用方法鏈結（Chaining）寫法實現。」

4. 語法解析

```
let fruits = ["蘋果", "香蕉", "橘子"]; // 宣告包含三個字串的陣列

fruits.push("草莓"); // 1. 在陣列尾端塞入 "草莓"
console.log(fruits); // 輸出 ["蘋果", "香蕉", "橘子", "草莓"]

// 2. 使用 forEach 走訪陣列
fruits.forEach(function(item, index) {
  console.log("第 " + (index + 1) + " 名水果是：" + item);
});

// 3. 使用 filter 篩選出字數大於兩個字的商品
let longNameFruits = fruits.filter(item => item.length > 2);
console.log(longNameFruits); // 輸出符合條件的陣列
```

```
// 陣列進階運算實作
let cart = [
  { name: "日本蘋果", price: 120 },
  { name: "鮮乳一杯", price: 85 },
  { name: "頂級和牛", price: 1200 },
  { name: "家庭號抽取衛生紙", price: 240 }
];

// 1. 篩選出價格小於 500 元的親民好物
let budgetFriendly = cart.filter(item => item.price < 500);
console.log("預算內商品:", budgetFriendly);

// 2. 將所有商品的名稱與價格，格式化成字串陣列
let labelList = cart.map(item => `${item.name} (${item.price} 元)`);
console.log("商品標籤清單:", labelList);
```

5. AI 常見錯誤

- 直接修改原始陣列 (Mutate)：AI 在進行資料處理時，有時會混淆「會修改原陣列的方法」(如 push/splice) 與「不會修改原陣列、而是回傳新陣列的方法」(如 map/filter)，導致原始資料被意外破壞，畫面狀態出現混亂。
- 解決方案：在提示詞中指明：「請採用不可變資料 (Immutable Data) 的觀念，使用 map 或 filter 等純函式方法，不要直接修改原始資料陣列。」

八、動態增刪網頁元素

1. 核心觀念

- (1) 動態建立：document.createElement("標籤名稱") 可以在記憶體中生成 HTML 元素。
- (2) 節點掛載：新建的元素必須掛載到現有的 DOM 樹 (如：parentElement.appendChild(childElement)) 中，使用者才看得到。

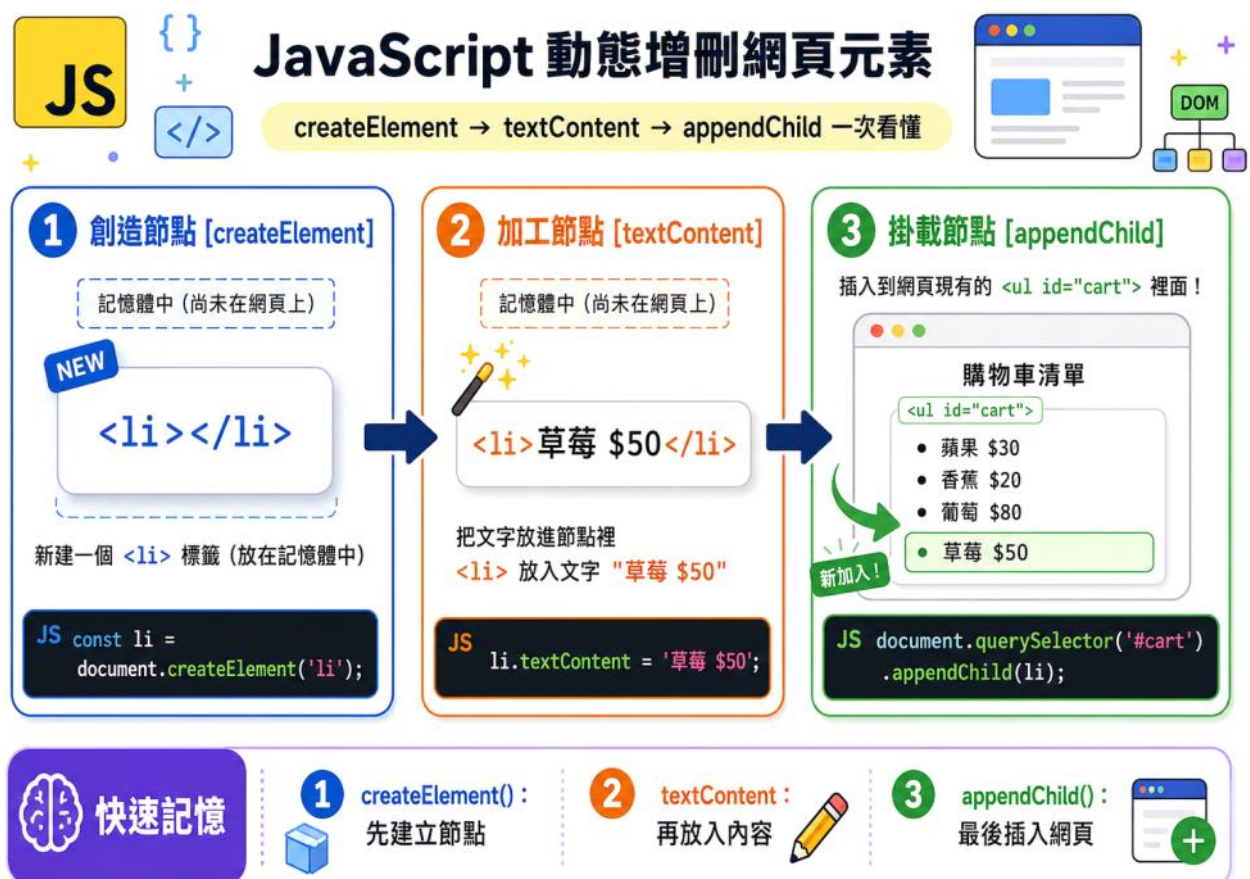
(3) XSS 安全性漏洞：

- 1) **innerHTML**：會解析 HTML 標籤。如果直接渲染使用者輸入的內容，容易遭受跨站腳本攻擊 (XSS)。
- 2) **textContent**：只會將內容視為純文字，非常安全。

2. 生活化案例

動態增刪元素就像是「樂高積木」：

- (1) **document.createElement**：在工廠裡製造一塊「新的紅色積木」。它目前還在你的手裡，還沒有裝在模型上。
- (2) **appendChild**：你把這塊新的紅色積木，「插在」原本已經蓋好的城堡上面。
- (3) **remove**：把城堡上的某個舊積木拆除扔掉。
- (4) 安全警告：innerHTML 就像是用怪手直接往城堡裡倒泥沙（可能帶有病毒/惡意程式碼）；textContent 則是小心翼翼地寫上純文字。



3. Prompt 範例

初階	「請寫出 JavaScript，功能是：當使用者點擊網頁上的一個按鈕時，網頁會動態在 id 為 todo-list 的無序清單 (ul) 裡面，新增一個寫有『買牛奶』的清單項目 (li)。」
進階	「請寫出一段安全且有效率的 JavaScript 程式碼。當使用者點擊按鈕時，會動態在 id 為 container 的 div 中新增一個卡片元件 (Card)。卡片元件包含：一個 h3 標題、一個 p 描述文字、和一個『刪除』按鈕。點擊該『刪除』按鈕時，必須能精確

4. 語法解析

```
// 1. 在 JS 中建立一個新的 <li> 標籤
const newLi = document.createElement("li");

// 2. 幫這個 <li> 放入文字內容
newLi.textContent = "新鮮芒果 $80";

// 3. 找到網頁上已存在的 <ul>，將這個 <li> 塞到裡面去
const list = document.querySelector("#product-list");
list.appendChild(newLi); // 掛載成功！畫面上會立刻多出一行
```

```

<!DOCTYPE html>
<html lang="zh-TW">
<head>
  <meta charset="UTF-8">
  <title>動態清單管理</title>
</head>
<body>
  <h3>購物備忘錄</h3>
  <input type="text" id="item-input" placeholder="想買什麼?">
  <button id="add-btn">新增</button>

  <ul id="shopping-list">
    <!-- 動態新增的項目會放到這裡 -->
  </ul>

  <script>
    const addBtn = document.querySelector("#add-btn");
    const itemInput = document.querySelector("#item-input");
    const shoppingList = document.querySelector("#shopping-list");

    addBtn.addEventListener("click", function() {
      const textValue = itemInput.value.trim();
      if (textValue === "") return; // 空白防護

      // 1. 建立一個 li 項目
      const li = document.createElement("li");
      li.textContent = textValue + " "; // 安全設定純文字

      // 2. 建立刪除按鈕
      const deleteBtn = document.createElement("button");
      deleteBtn.textContent = "刪除";

      // 3. 給刪除按鈕裝上點擊事件：當被點擊時，自己刪除這個 li
      deleteBtn.addEventListener("click", function() {
        li.remove();
      });

      // 4. 組裝與掛載
      li.appendChild(deleteBtn);
      shoppingList.appendChild(li);

      // 5. 清空輸入框
      itemInput.value = "";
    });
  </script>
</body>
</html>

```

5. AI 常見錯誤

- 盲目使用 innerHTML 引發 XSS：AI 為了圖方便，常常直接寫 `element.innerHTML = '<li>' + userInput + '</li>'`，這在前端開發中是極大的安全隱憂。
- 解決方案：在提示詞中加上安全限制：「在動態渲染文字時，請一律使用 `textContent` 或

document.createElement 進行安全處理，禁止直接使用未過濾的 innerHTML 拼接使用者輸入。

九、瀏覽器本地儲存 (LocalStorage)

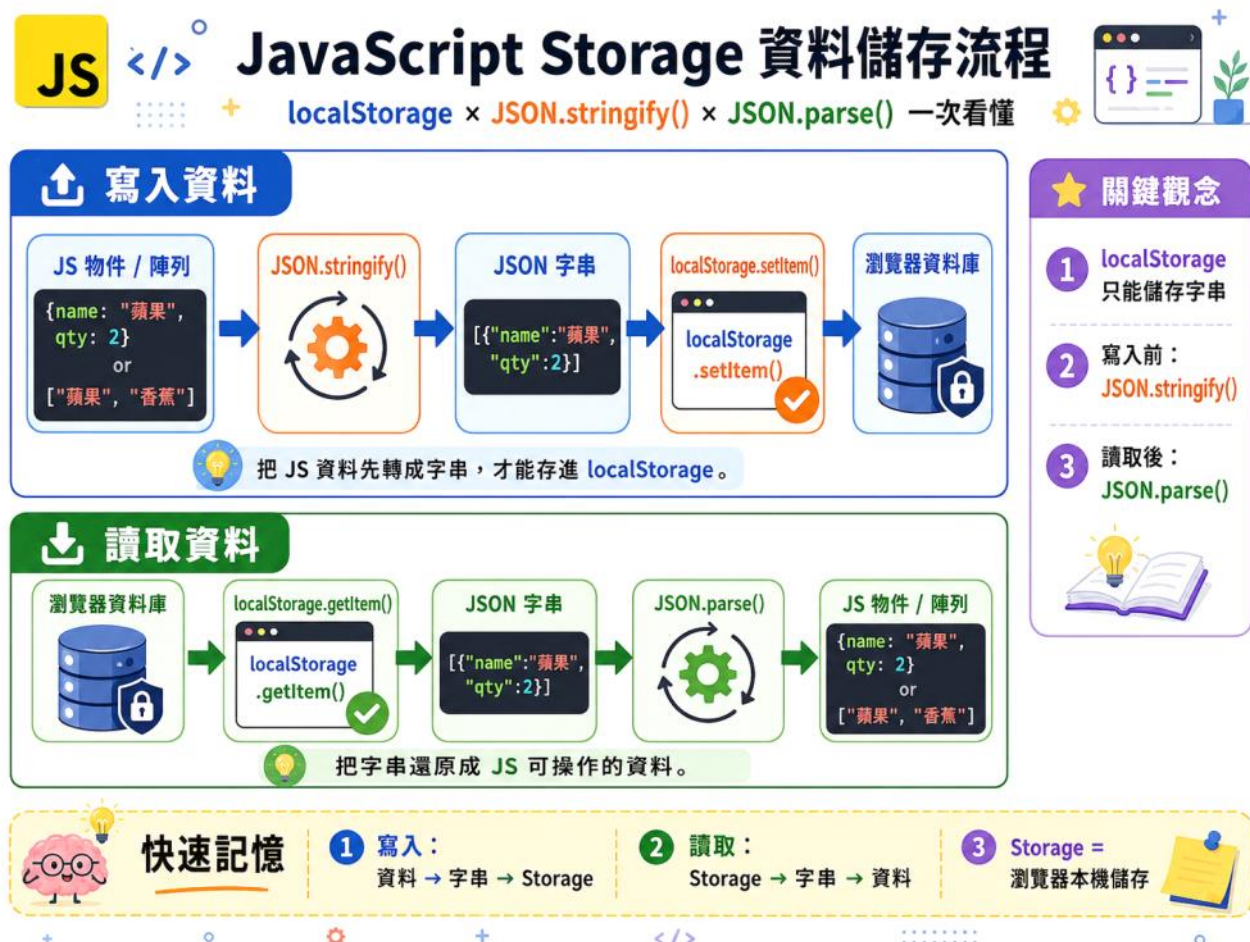
1. 核心觀念

- (1) **Key-Value 鍵值對結構**：LocalStorage 儲存方式為「一把鑰匙 (Key) 對應一個箱子 (Value)」。
- (2) **純字串限制**：LocalStorage 的箱子只收純文字 (String)。如果你把一個陣列 [1, 2] 或物件直接丟進去，它會被強制轉型成毫無用處的 "[object Object]"。
- (3) **JSON 工具轉換組合**：
 - 1) **寫入前 (序列化)**：用 JSON.stringify(陣列或物件) 把它壓扁成 JSON 格式的文字。
 - 2) **讀取後 (反序列化)**：用 JSON.parse(字串) 把它還原回原本的 JavaScript 陣列或物件。

2. 生活化案例

旅館儲物櫃 vs. 自家保險箱：

- (1) **Session 變數 (暫存記憶體)**：就像你在捷運站租的臨時置物櫃。一整天過後 (重新整理或關閉分頁)，櫃子就會被強制清空 (資料不見)。
- (2) **LocalStorage (本地儲存)**：就像是你在家裡的專屬密碼保險箱。除非你手動拿鑰匙清除 (清空瀏覽器快取或呼叫程式碼)，否則不論隔了多久，重新開機後保險箱裡的財寶 (資料) 依然會在那裡！



3. Prompt 範例

初階	「請告訴我如何在 JavaScript 當中使用 LocalStorage。請給出最基礎的『寫入數據、讀取數據、移除單一數據、清除所有數據』的程式碼示範。」
進階	「我需要做一個儲存使用者喜好設定 (Preference) 的功能，包含使用者主題 (theme: 'dark') 與通知開關 (notify: true)。請用 JavaScript 寫出一段防呆且安全的方法：在寫入 LocalStorage 前使用 JSON.stringify 轉換，並在讀取時用 JSON.parse 還原。如果 LocalStorage 裡尚未儲存該資料，請回傳一個預設的主題物件，避免程式碼拋出 null 錯誤。」

4. 語法解析

```
// 1. 存入單一基礎字串
localStorage.setItem("username", "小明");

// 2. 取出單一基礎字串
let name = localStorage.getItem("username");
console.log(name); // 輸出 "小明"

// 3. 處理複雜陣列並存入
let todos = ["買牛奶", "洗衣服"];
// ❌ 錯誤: localStorage.setItem("todos", todos); -> 存進去會變成 "買牛奶,洗衣服"
let jsonString = JSON.stringify(todos); // 轉成 '["買牛奶","洗衣服"]'
localStorage.setItem("myTodoList", jsonString);

// 4. 取出陣列並還原
let savedString = localStorage.getItem("myTodoList");
let savedTodos = JSON.parse(savedString); // 還原成真正的陣列結構
console.log(savedTodos[0]); // 輸出 "買牛奶"
```

5. AI 常見錯誤

- 未提供 null 的安全防禦：當第一次載入網頁時，LocalStorage 裡絕對是空的 (也就是 null)，如果 AI 寫完 `JSON.parse(localStorage.getItem("key"))` 沒有加上 `|| {}` 或預設值防呆，程式碼就會因為讀取 null 的屬性而瞬間崩潰 (Crash)。
- 解決方案：要求 AI 寫取出的程式碼時，必須一律套用邏輯短路求值。例如：`let data = JSON.parse(localStorage.getItem("key")) || {};`

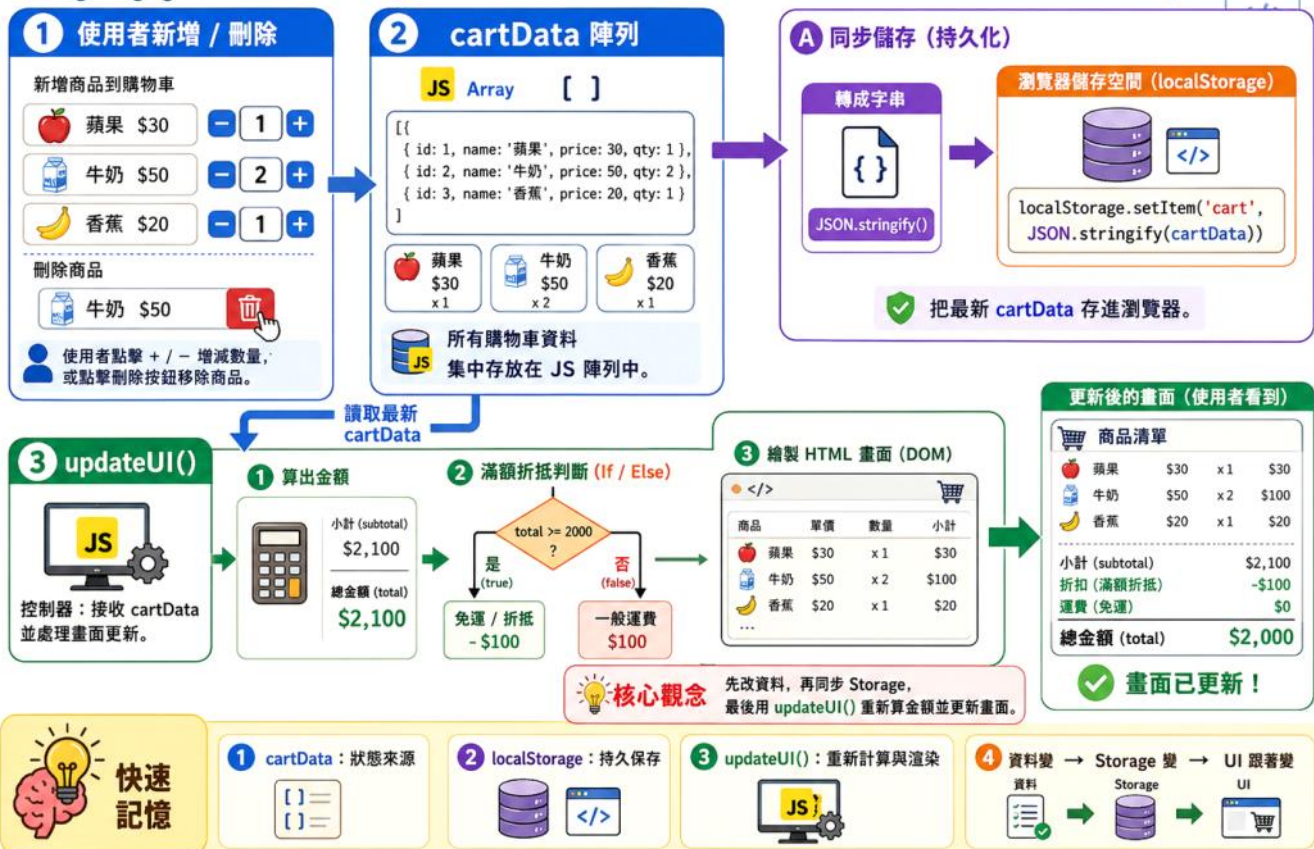
十、綜合專案：超市結帳系統



超市結帳系統 — 永續狀態儲存資料流

cartData × localStorage × updateUI() × If/Else × DOM 一次看懂

JS



Prompt 範例

初階	「請幫我寫一個超市結帳系統的 HTML，需要有一個商品名稱、一個價格輸入框、一個新增按鈕，和一個可以顯示商品跟總金額的區域。請加上基礎、簡潔的 CSS 美化樣式。」
進階	「我需要做一個超市結帳系統的 JavaScript。功能規格： 1. 全域變數 cartData 在網頁初始化時，必須先讀取 LocalStorage 鍵值 "supermarketCart" 的資料，若無資料則給予空陣列。 2. 當點擊『新增』按鈕，將商品名稱和價格包裝成物件並推入 cartData，其中價格需轉為數字，並加入唯一 ID (例如 Date.now())。 3. 每次資料變動 (新增或刪除) 時，都要呼叫 saveToLocalStorage() 函式與 updateUI()。 4. updateUI() 負責先清空畫面，再用 forEach 和 createElement 渲染商品清單，每一列商品後方要有一個『刪除』按鈕，並能正確透過 ID 呼叫 deleteItem。